15 – Cost control

AI Training for Beginners

Self-study training: use AI/LLMs safely in everyday life

Version 1.0 • As of: October 2026

File: `15_cost_control.md`

https://github.com/mgunthner/AI-Training

License: CC BY-NC-SA 4.0

Contents

15 – Cost control **1**
Key points 1
Example 1
Do / Don’t 2
Safety note 2
Mini checklist 2
Exercise (5 min.) 2
Next steps 3

15 – Cost control

Definition: Cost control means: knowing how AI use is paid for (free limits, subscription, API per token), where costs arise, and how you keep them under control.

> **Remember:** Long inputs, long answers, and reasoning cost money – or your limit.

Key points

- **Three payment models:**
  - **Free plan:** no monetary costs, but usage limits (messages per time period)
  - **Subscription:** higher limits and more features for a monthly price
  - **API:** payment per token – both incoming and outgoing (topic 02)
- **Cost drivers:** a large model instead of a small one, long inputs (the entire chat history!), long answers, reasoning mode (topic 10), file and image processing, agent loops (every step costs).
- **Saving without losing quality:** a small model for routine work, short context with only what’s relevant, a new chat instead of an endless history, limiting response length, summarizing repeated content instead of pasting it in again.
- **The basic idea behind caching and batch:** Repeated content (e.g. the same context) can be cached and processed more cheaply, and bulk jobs can be handled at lower rates. Details depend on the provider.
- **Protect yourself:** set a budget/spending limit (where possible), set up warning thresholds, use a separate key with its own limit for each use case, review usage regularly.
- **Limits, not panic:** On the free plan, exceeding the limit costs you time, not money. Still: use reasoning mode and large models deliberately, and your limit will last longer.
- **Prices change:** Always check your provider’s current pricing page.

Example

You want to summarize and answer 20 customer messages per week – numbers and names are made up:

Variant	Approach	Tendency
1	One chat, all 20 emails one after another in reasoning mode	History grows, every call re-processes everything, limit runs out fast
2	A new chat per email, small model, short prompt	Minimal context, low costs
3	Variant 2, but with the instruction stored permanently as a system prompt	Typing saved, similarly low costs

Variant 2 or 3 completes the same task for a fraction of the tokens – at roughly the same quality, because the task is routine.

Do / Don’t

Do	Don’t
Use a small model for routine tasks	Do everything with the largest reasoning mode
Start a new chat when the old one gets long	Keep endless chats and drag everything along
Review limits and usage regularly	Ignore limits and hope for the best on the invoice
One key per project with its own limit	Use one key for everything

Safety note

Surprisingly high API costs are often a sign of a leaked API key. If you see unexpected charges, check the usage history first and revoke the key if in doubt (topic 14).

Mini checklist

- ☐ I know the three payment models and what mine costs.
- ☐ I can name three cost drivers.
- ☐ I set limits and review usage regularly.
- ☐ When costs look unexpected, I check the usage history first.

Exercise (5 min.)

Rough estimate: a routine task (summarizing an email) takes you about 500 input tokens and 200 output tokens per operation. At 100 operations per month, that’s around 70,000 tokens. What would save the most: shorter context, shorter answers, or fewer operations? Do a rough calculation and assume output tokens are more expensive than input tokens.

Next steps

- Extra exercises: `02_extra_exercises/15_extra_exercises`
- Test: Test D (Technology & costs)
- Related: 02 tokens & context, 03 model selection, 10 reasoning effort, 14 API basics

As of: October 2026